Keyword	Meaning / Use
`abstract`	Defines a class or method that **cannot be instantiated directly**. Must be implemented in a subclass.
`as`	**Typecast** an object to a different type.
`assert`	Used for **debug checks**. Example: `assert(x > 0);` will throw an error if false.
`async`	Marks a function as **asynchronous**, allowing `await` inside.
`await`	Waits for a **Future** to complete.
`break`	Exits a **loop** or **switch** early.
`case`	Defines a **branch** in a `switch` statement.
`catch`	Catches an **exception** in a `try-catch` block.
`class`	Declares a **class**.
`const`	Declares a **compile-time constant** value.
`continue`	Skips **current loop iteration** and continues with next.
`covariant`	Used in **type checking in class overrides**. Advanced topic.
`default`	Default case in a `switch` statement.
`deferred`	For **lazy loading** of libraries.
`do`	Starts a **do-while loop**.
`dynamic`	Variable whose **type is determined at runtime**.
`else`	Defines code for **false condition** in `if` statements.
`enum`	Defines a set of **named constant values**.
`export`	Makes a **library available** to other files.
`extends`	Defines a **subclass** of another class.
`extension`	Adds methods to existing classes **without modifying them**.
`external`	Marks a **function or variable implemented outside Dart**.
`factory`	Defines a **factory constructor** that can return new or existing instances.
`final`	Variable **can be set only once**.
`finally`	Code that **always runs after try-catch**, whether an exception occurred or not.
`for`	Starts a **for loop**.
`Function`	Type for **function references**.
`get`	Defines a **getter** method to read a property.
`if`	Starts a conditional check.
`implements`	Declares that a class **implements an interface**.
`import`	Loads another **library or file**.
`in`	Checks **membership** in loops or collections.
`interface`	Declares an **interface** (Dart does not use this keyword anymore).
`is`	Checks if an object **is of a certain type**.
`late`	Marks a variable that will be **initialized later**, not at declaration.
`library`	Declares a **library name** for organization.

Keyword	Meaning / Use
`mixin`	Reusable **class code** that can be added to other classes.
`new`	Creates a new **object instance** (optional in modern Dart).
`null`	Represents a **null value**.
`on`	Used in **exception handling** to catch specific exception types.
`operator`	Overloads an **operator** in a class (like `+`, `-`).
`part`	Defines **part of a library**.
`required`	Marks a **named parameter as required** in functions or constructors.
`return`	Returns a value from a **function**.
`set`	Defines a **setter** method to assign a value.
`static`	Class-level **variable or method**, not tied to an object.
`super`	Refers to the **parent class**.
`switch`	Starts a **switch-case statement**.
`this`	Refers to the **current instance of the class**.
`throw`	Throws an **exception**.
`try`	Starts a **try-catch block** for exceptions.
`typedef`	Defines a **function type alias**.
`var`	Declares a variable with **inferred type**.
`void`	Function **returns nothing**.
`while`	Starts a **while loop**.
`with`	Adds a **mixin** to a class.
`yield`	Returns a value in a **generator function**.

Keyword / Concept	Meaning
`Widget`	Base class for **all Flutter UI elements**.
`StatelessWidget`	Widget **that doesn't change state** after creation.
`StatefulWidget`	Widget **that can change state** dynamically.
`build(BuildContext context)`	Function called to **render the widget**.
`setState()`	Used in a StatefulWidget to **update the UI**.
`MaterialApp`	Root widget for **Material Design apps**.
`Scaffold`	Provides a **basic visual layout structure** (app bar, body, drawer).
`Container`	A **box** that can hold other widgets and styling.
`Column`	Layout widget **stacking children vertically**.
`Row`	Layout widget **stacking children horizontally**.
`Padding`	Adds **space inside a widget**.
`Expanded`	Makes a widget **take remaining space** in Row or Column.
`Center`	Centers a child widget.
`Navigator`	Manages **screen navigation** (push, pop routes).
`Text`	Displays **text on screen**.
`TextField`	Input field for **user text input**.
`ListView`	Scrollable list of widgets.
`FutureBuilder`	Builds a widget based on **async data**.
`StreamBuilder`	Builds a widget based on **streaming data**.
`setState`	Updates the **state of a widget** and rebuilds UI.
`initState`	Runs **once when StatefulWidget is created**, used for initialization.
`dispose`	Cleans up resources when **widget is removed**.